

Homework 4

Haozheng Wang

November 2025

1 Basic Stereo Matching Algorithm

1.1 Disparity Map Computation

The idea of this implementation is straightforward. First, it is quite natural to pad the images before extracting the windows. Then, for each pixel in the first image, we examine all pixels on the scan line (i.e. the same row) in the second image and pick the best match.

To be specific, a square window with a size of *window_size* is created around this pixel in the first image. Windows with the same size are generated around those pixels on the scan line in the second image. For each pair of windows, we calculate window differences and select the best pair that minimize this result, thus obtaining the best disparity. It is noteworthy that for *normalized_correlation*, our objective is to find out the best pair that maximize the correlation, and the code need to be adjusted accordingly.

Acceleration has been achieved by ignoring pixels with x-coordinates greater than the pixel being inspected in the first image, as we search for the best disparity within (*min_disparity*, *max_disparity*). And in order to record the running time, we mark the timing at the beginning and end of this function using *time.time()* and calculate the time difference.

```
1 def task1_compute_disparity_map_simple(  
2     ref_img: np.ndarray,          # shape (H, W)  
3     sec_img: np.ndarray,          # shape (H, W)  
4     window_size: int,  
5     disparity_range: Tuple[int, int], # (min_disparity,  
6         max_disparity)  
7     matching_function: str # can be 'SSD', 'SAD', '  
8         normalized_correlation'  
9 ):  
10     time_start = time.time()  
11  
12     disparity_map = np.zeros_like(ref_img)  
13     H, W = ref_img.shape  
14     radius = window_size // 2  
15     min_disparity, max_disparity = disparity_range  
16     # 0. Pad ref_img and sec_img for window extraction  
17     ref_img = np.pad(ref_img, ((radius, radius), (radius, radius)),  
18         'edge')
```

```

16 sec_img = np.pad(sec_img, ((radius, radius), (radius, radius)),
17                    'edge')
18 # 1. For each row, scan all pixels in that row
19 for row in range(radius, radius + H):
20     for col in range(radius, radius + W):
21         # 2. Generate a window for each pixel in ref_img
22         ref_window = ref_img[row-radius:row+radius, col-radius:
23                             col+radius]
24         best_disparity = min_disparity
25         if matching_function == 'SSD' or matching_function == '
26             SAD':
27             min_window_difference = float('inf')
28             elif matching_function == 'normalized_correlation':
29                 max_normalized_correlation = -1
30             # 3. Search for a disparity (d) within (min_disparity,
31             max_disparity) in sec_img
32             for d in range(min_disparity, max_disparity + 1):
33                 if col - d < radius:
34                     break
35                 sec_window = sec_img[row-radius:row+radius, col-d-
36                                     radius:col-d+radius]
37             # 4. Select the best disparity that minimize window
38             difference between ref_img[row, col] and
39             sec_img[row, col - d]
40             if matching_function == 'SSD': # sum of squared
41                 differences
42                 cur_window_difference = np.sum((ref_window -
43                                                 sec_window) ** 2)
44                 if cur_window_difference <
45                     min_window_difference:
46                     min_window_difference =
47                         cur_window_difference
48                     best_disparity = d
49             elif matching_function == 'SAD': # sum of absolute
50                 differences
51                 cur_window_difference = np.sum(np.abs(
52                     ref_window - sec_window))
53                 if cur_window_difference <
54                     min_window_difference:
55                     min_window_difference =
56                         cur_window_difference
57                     best_disparity = d
58             elif matching_function == 'normalized_correlation':
59                 cur_normalized_correlation = \
60                     np.sum((ref_window - np.mean(ref_window)) *
61                             (sec_window - np.mean(sec_window))) /
62                     \
63                     (np.sqrt(np.sum((ref_window - np.mean(
64                         ref_window)) ** 2)) *
65                      np.sqrt(np.sum((sec_window - np.mean(
66                          sec_window)) ** 2)))
67                 if cur_normalized_correlation >
68                     max_normalized_correlation:
69                     max_normalized_correlation =
70                         cur_normalized_correlation
71                     best_disparity = d
72             disparity_map[row-radius, col-radius] = best_disparity

```

```

52 |
53 | time_end = time.time()
54 | with open("task_1_time_record.txt", 'a', encoding='utf-8') as f
55 |     :
56 |     f.write(f"Task 1 running time (window_size={window_size},
        disparity_range={disparity_range}, matching_function={
        matching_function}): {time_end - time_start}\n")
        return disparity_map

```

1.2 Hyperparameter Settings and Report

1.2.1 How does the running time depend on window size, disparity range, and matching function?

- **Window size:** the larger the window size, the longer the running time; however, their relationship is not that proportional.

Task 1 running time (window_size=2, disparity_range=(0, 20), matching_function=SSD): 9.462082386016846

Task 1 running time (window_size=5, disparity_range=(0, 20), matching_function=SSD): 9.544005393981934

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=SSD): 9.898839235305786

Task 1 running time (window_size=20, disparity_range=(0, 20), matching_function=SSD): 11.067171573638916

Task 1 running time (window_size=2, disparity_range=(0, 20), matching_function=normalized_correlation): 69.91195583343506

Task 1 running time (window_size=5, disparity_range=(0, 20), matching_function=normalized_correlation): 85.43727374076843

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=normalized_correlation): 77.32371854782104

Task 1 running time (window_size=20, disparity_range=(0, 20), matching_function=normalized_correlation): 100.02356052398682

- **Disparity range:** the larger the disparity range, the longer the running time, and their relationship is roughly proportional.

Task 1 running time (window_size=10, disparity_range=(0, 5), matching_function=SSD): 2.9466404914855957

Task 1 running time (window_size=10, disparity_range=(0, 10), matching_function=SSD): 5.7782301902771

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=SSD): 10.4430570602417

Task 1 running time (window_size=10, disparity_range=(0, 5), matching_function=normalized_correlation): 22.945095777511597

Task 1 running time (window_size=10, disparity_range=(0, 10), matching_function=normalized_correlation): 41.934810638427734

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=normalized_correlation): 75.06681418418884

- **Matching function:** the running times of SSD and SAD are approximately the same, while the running time of normalized correlation (a.k.a. ZNCC) is significantly longer.

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=SSD): 10.057991027832031

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=SAD): 9.974781036376953

Task 1 running time (window_size=10, disparity_range=(0, 20), matching_function=normalized_correlation): 77.68101572990417

1.2.2 Which window size works the best for different matching functions?

For this part of experiment, we set disparity_range=(0, 20) uniformly.

For SSD, effects of different window sizes are shown below. When the window size is relatively small, more details are retained while the disparity map looks more noisy. And when the window size grows larger, smoother results with less noise (also with less details, though) are obtained. Overall, window_size=10 works the best for SSD.

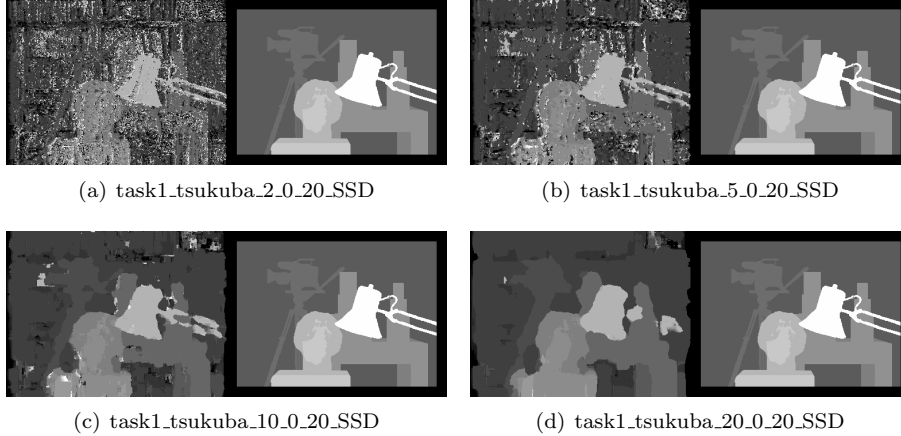


Figure 1: Different Window Sizes for SSD

For SAD, the speed of disparity map blurring turns out to be much rapid than expected, so we have to narrow down the scope of *window_size*. Overall, *window_size*=5 works best for SAD.

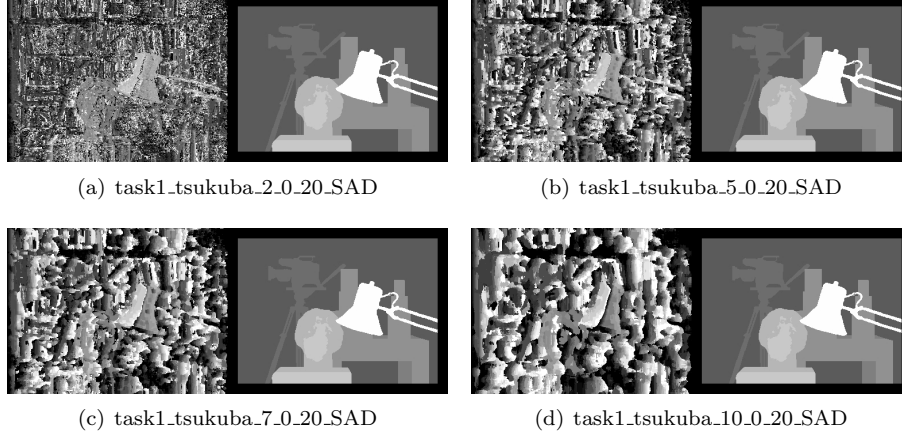


Figure 2: Different Window Sizes for SAD

For normalized_correlation, the trend of disparity map quality as *window_size* grows larger shows consistency with the previous two matching functions. However, the variance seems to be larger than SSD and SAD, since the map turns out to be more noisy when *window_size* is relatively small and more blurry when *window_size* grows larger. Overall, *window_size*=10 works best for normalized_correlation.

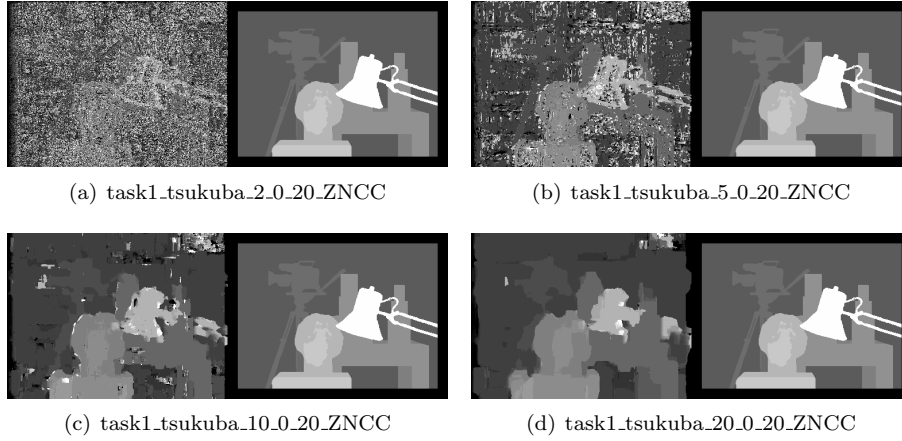


Figure 3: Different Window Sizes for ZNCC

1.2.3 What is the maximum disparity range that makes sense for the given stereo pair?

For this part of experiment, we set `window_range=10` and choose SSD as the matching function. As the maximum disparity range expands, the brightness of the disparity map gradually declines, and the maximum disparity range that makes sense for the given stereo pair is approximately (0, 50).

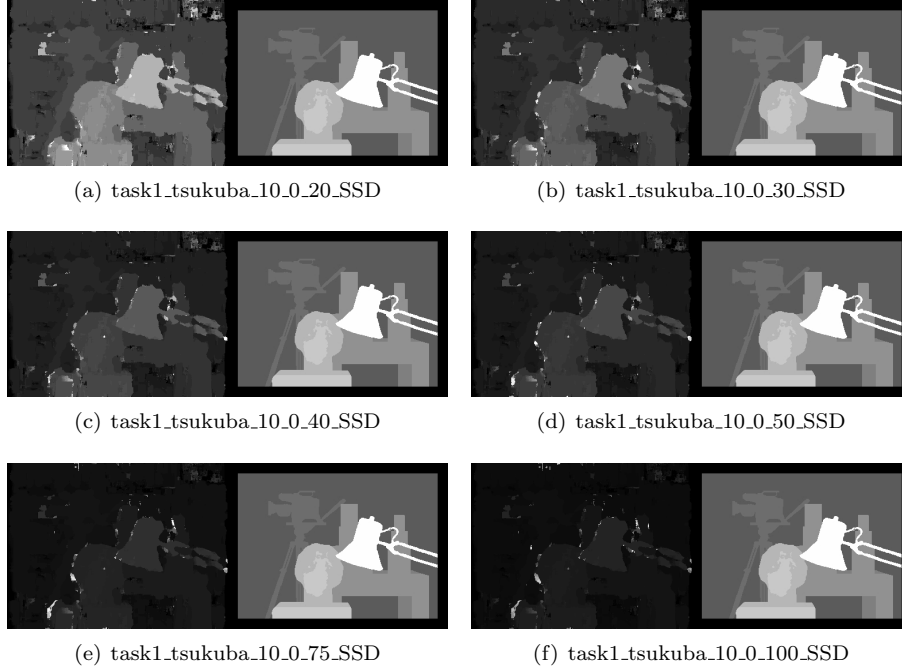


Figure 4: Different Disparity Range for SSD

1.2.4 Which matching function may work better for the given stereo pair?

It can be seen clearly that SSD works better for the given stereo pair, as shown in 1.2.2.

1.3 Discussion

1.3.1 Discuss the trade-offs between different hyperparameters on quality and time

- **Window size:** As the window size increases, the disparity map becomes smoother but with fewer details, so a balance between details and smoothness need to be achieved. The runtime increases with the window size, though the relationship is not directly proportional.

- **Disparity range:** As the disparity range extends, the brightness of the disparity map comes down and more outliers may occur. As a result, the disparity range should be kept within a small but sufficient range enabling correct matching. The running time increases approximately proportional with the disparity range.
- **Matching function:** ZNCC takes a longer running time than SSD and SAD with worse results, and SAD works much worse than SSD. So it is a natural idea to take SSD as our first choice.

1.3.2 Choose the best hyperparameters and show the corresponding disparity map

We choose `window_size=10`, `disparity_range=(0, 20)` and `matching_function='SSD'`.



Figure 5: Disparity Map with the Best Hyperparameters

1.3.3 Compare the best disparity map with the ground truth map, discuss the differences and limitations of basic stereo matching

Compared to the ground truth map, the best disparity map has more noise, and the edge of objects are not that smooth. Moreover, some details in the map are lost severely.

Basic stereo finds the best match for each window independently, without considering local constraints such as matching order and nearest neighbor principle. In addition, if there are half occlusions, textureless regions, repeated patterns and specularities, the risk of mismatching significantly increases.

2 Depth from Disparity

2.1 Pointcloud Visualization

The implementation of `task2_compute_depth_map()` can be quite straightforward using the formula $z = \frac{Bf}{x-x'}$ and Numpy operations.

Note: When a disparity in the disparity map is less or equal to zero, we set the depth at the corresponding position in the depth map to zero. *"Note that a disparity less or equal to zero should be ignored (set to zero)"* seems sort of misleading.

```
1 def task2_compute_depth_map(disparity_map, baseline, focal_length):
2     mask = disparity_map > 0
3     depth_map = np.zeros(disparity_map.shape)
4     depth_map[mask] = focal_length * baseline / disparity_map[mask]
5     return depth_map
```

For the implementation of `task2_visualize_pointcloud()`, we first exclude some outliers by discarding depths that deviates from the average depth by more than 2 sigma, with a mask array created at the same time. We use `np.indices()` to help generate the x and y coordinates, and use `np.hstack()` to create the points array combining x, y and z coordinates (just like what we have done in previous assignments to convert Cartesian coordinates to homogeneous coordinates!). The colors array can be calculated straightforwardly by using the mask array generated above.

Note: In the sentence `depth_map=depth_map/np.max(depth_map)*200.0`, the scaling constant 200.0 is determined by nothing but empirical experiments.

```
1 def task2_visualize_pointcloud(
2     ref_img: np.ndarray, # shape (H, W, 3)
3     disparity_map: np.ndarray, # shape (H, W)
4     save_path: str = 'output/task2_tsukuba.ply'
5 ):
6     baseline = 10
7     focal_length = 10
8     depth_map = task2_compute_depth_map(disparity_map, baseline,
9                                         focal_length)
10    # cut some outliers for better performance
11    depth_map_tilda = depth_map.copy()
12    mean = np.mean(depth_map_tilda)
13    std = np.std(depth_map_tilda)
14    depth_map_tilda = np.abs((depth_map_tilda - mean) / std)
15    depth_map[depth_map_tilda > 2] = 0
16    mask = depth_map > 0
17    y_coors, x_coors = np.indices(depth_map.shape)
18    depth_map = depth_map / np.max(depth_map) * 200.0
19    # Points
20    points = np.hstack((x_coors[mask].reshape(-1, 1), y_coors[
21                        mask].reshape(-1, 1), depth_map[mask].reshape(-1, 1)))
22    # Colors
23    colors = ref_img[mask] / 255.0
```

```

22 # Save pointcloud to ply file
23 pointcloud = trimesh.PointCloud(points, colors)
24 os.makedirs(os.path.dirname(save_path), exist_ok=True)
25 pointcloud.export(save_path, file_type='ply')

```

2.2 Reports

The results of pointcloud visualization are displayed below:

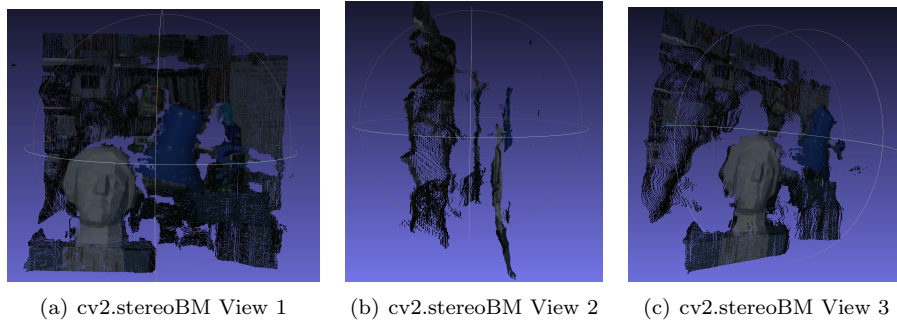


Figure 6: cv2.stereoBM Views

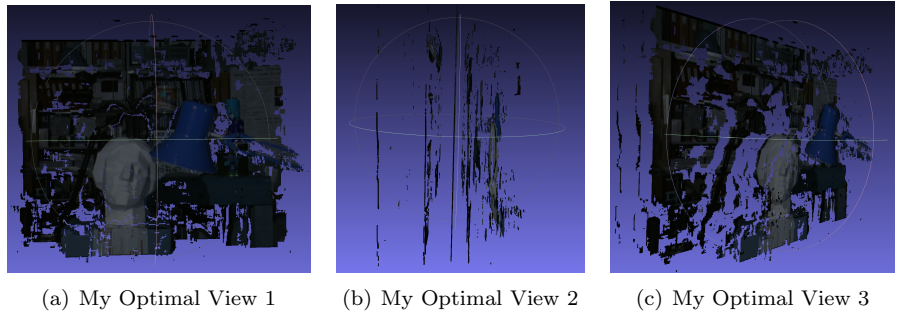


Figure 7: My Optimal Views

Compared with the pointcloud generated by **cv2.stereoBM**, the pointcloud generated by my optimal settings is not so smooth and continuous. In addition, there are still a proportion of outliers that have not been removed.

3 Stereo Matching with Dynamic Programming

3.1 Algorithm Implementation

Before we implement the main body of this algorithm, it is necessary to make some preparations, including padding the image so that we can extract windows without considering out-of-index issues.

```
1 H, W = ref_img.shape
2 window_size = 3
3 radius = window_size // 2
4 max_disparity = 16
5 occlusionConstant = 256.0
6 disparity_map_dp = np.zeros((H, W))
7 ref_img = np.pad(ref_img, ((radius, radius), (radius, radius)), '
    edge')
8 sec_img = np.pad(sec_img, ((radius, radius), (radius, radius)), '
    edge')
```

We create matrices e , C and B for each row of the image pair, whose meanings are exactly the same as those in the slides. Just bear in mind that the row-coordinates corresponds to indexes in the right scanline and the column-coordinates corresponds to indexes in the left scanline for e , C and B .

```
1 for row in range(radius, radius + H):
2     e = np.full((W, W), float('inf')) # the matching cost of each
    match
3     C = np.full((W, W), float('inf')) # C[i, j]: the minimum cost
    to the match (i, j)
4     B = np.zeros((W, W)) # leverage the matrix B to recover the
    path via backtracing
```

Our first substantive work is to compute matrix e according to SSD. We use two loops in our implementation, and throw out positions whose row-coordinate is larger than its column-coordinate (i.e., $x' > x$ is not allowed) and whose column-coordinate is larger than its column-coordinate by more than max_disparity (i.e., $x - x' > \text{max_disparity}$ is not allowed). These accelerations are necessary as they reduce time complexity from $O(W \times W)$ to $O(W \times \text{max_disparity})$.

```
1 # Compute matrix e according to SSD
2 for i in range(radius, radius + W):
3     sec_window = sec_img[row - radius:row + radius, i - radius:i +
    radius]
4     for j in range(i, min(i + max_disparity, radius + W)):
5         ref_window = ref_img[row - radius:row + radius, j - radius:j +
    radius]
6         e[i - radius, j - radius] = np.sum((ref_window - sec_window
    ) ** 2)
```

Next, we initialize the first row for matrix C and B using a naive method of initialization, which was also adopted in the original paper ¹.

```

1  # Initialize the first row
2  C[0, 0] = min(e[0, 0], occlusionConstant)
3  B[0, 0] = -1
4  for j in range(1, W):
5      C[0, j] = j * occlusionConstant
6      B[0, j] = 3 # Occluded from the left

```

Then comes the absolute core part of this algorithm. Our implementation of dynamic programming is almost the same as that shown in the slides, but we restrict the computational scope following the discipline that we have proposed above.

```

1  # Fill the cost table with dynamic programming
2  for i in range(1, W):
3      for j in range(i, min(i + max-disparity, W)):
4          min1 = C[i-1, j-1] + e[i, j]
5          min2 = C[i-1, j] + occlusionConstant
6          min3 = C[i, j-1] + occlusionConstant
7          C[i, j] = min(min1, min2, min3)
8          if min1 == C[i, j]:
9              B[i, j] = 1 # Matched
10         elif min2 == C[i, j]:
11             B[i, j] = 2 # Occluded from the right
12         elif min3 == C[i, j]:
13             B[i, j] = 3 # Occluded from the left

```

With the assistance of matrix B, we are able to recover the path via backtracing, or to put it in our context, to build the disparity map.

```

1  # Recover the path via backtracing
2  i, j = W - 1, W - 1
3  while i >= 0 and j >= 0:
4      if B[i, j] == 1: # Matched
5          disparity_map_dp[row-radius, j] = j - i
6          i, j = i - 1, j - 1
7      elif B[i, j] == 2: # Occluded from the right
8          disparity_map_dp[row-radius, j] = -2
9          i = i - 1
10     elif B[i, j] == 3: # Occluded from the left
11         disparity_map_dp[row-radius, j] = -3
12         j = j - 1
13     elif B[i, j] == -1: #
14         disparity_map_dp[row-radius, j] = 0
15         break

```

¹Cox, Hingorani, Rao, Maggs, “A Maximum Likelihood Stereo Algorithm”, 1996

Last, we carry out occlusion filling in attempt to reduce the number of "bad" points. We fill in left occluded pixels with value from the nearest valid pixel preceding it in the scanline. Similarly, for right occluded, we look for valid pixel to the right.

```

1  # Occlusion Filling
2  for col in range(radius, radius + W):
3      if disparity_map_dp[row-radius, col-radius] == -2: #
4          Occluded from the right
5          for nearest_right_column in range(col, radius + W):
6              if disparity_map_dp[row-radius,
7                  nearest_right_column-radius] > 0:
8                  disparity_map_dp[row-radius, col-radius] =
9                      disparity_map_dp[row-radius,
10                         nearest_right_column-radius]
11                 break
12 if disparity_map_dp[row-radius, col-radius] == -3: #
    Occluded from the left
    for nearest_left_column in range(col, radius, -1):
        if disparity_map_dp[row-radius, nearest_left_column
            -radius] > 0:
            disparity_map_dp[row-radius, col-radius] =
                disparity_map_dp[row-radius,
                    nearest_left_column-radius]
            break

```

3.2 Report



Figure 8: Disparity Map

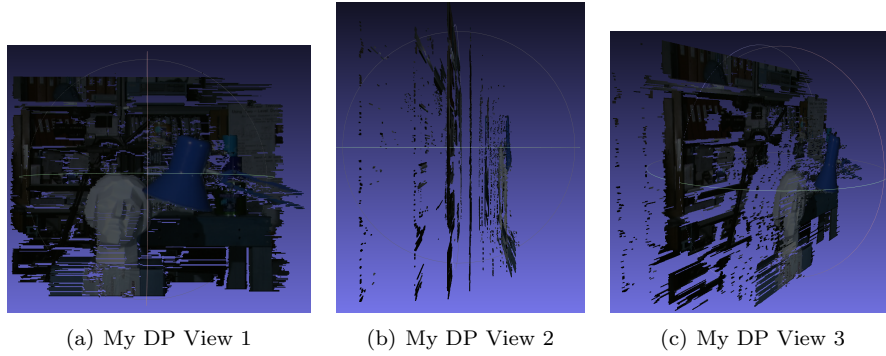


Figure 9: My DP Views

The running time of this algorithm in this problem is 10.46184229850769s. It should be mentioned that without those accelerations proposed in the previous subsection, the time cost would explode to more than two hundred seconds.

Compared with basic stereo matching algorithm, the results obtained through stereo matching with dynamic programming are blessed with better qualities. The edges of objects in the disparity map can match better with those in the ground truth. In addition, both smoothness and detail preservation are improved.

However, it cannot be ignored that the disparity map has a severe "wire drawing" effect. This abnormality could be attributed to the fact that the DP algorithm only minimize the matching cost within one row without considering interactions among adjacent rows.